

Lab Session 1: Raw GNSS measurements in Android devices

Laboratory on Post Processing of Raw Measurements | **LAB BRIEFING**

Simone Zocca, Andrea Nardin (Prof. Alex Minetto)

name.surname@polito.it



Motivation

Use of COTS devices for Precise Positioning

- Cost/form factor and size effective
- Processing capacity
- Benefits of embedded sensors and communication interfaces
- Going beyond single frequency
(e.g., Broadcom BCM47755 in Xiaomi Mi 8 Pro)

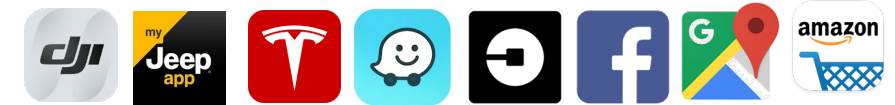
Availability of GNSS raw measurements from Android 7 onwards

- Opens the door to more advanced GNSS processing techniques
- Raw measurements allow to optimize the multi-GNSS solutions
[on-chip PVT can be optional]

Release of white paper for using GNSS Measurements on Android Devices

- Bridges the gap between GNSS experts and Android developers

Examples of mobile applications using location data



GNSS in COTS devices: pills of history



1999: Benefon Esc



May 2018: Xiaomi Mi 8 Pro



March 2004: TomTom



Google Maps

February 2005: Google Maps

- 1999 ● Mobile phone manufacturer Benefon launched the **first commercially-available GPS phone**, a safety phone called the Benefon Esc! The GSM phone was sold mainly in Europe, but many other GPS-enabled mobile phones would follow.
- May 2000 ● Switch-off of the Selective Availability (SA)
- 2001 ● As GPS receiver technology got much smaller and cheaper, private companies began pumping out **personal GPS products**, like the in-car navigation devices from Tom Tom and Garmin.
- 2006 ● Google patented the SUPL service (US7714779B2) to provide “real-time” long-term-orbit **ephemeris data to mobile Android** devices by overcoming the need of demodulating the whole navigation message (almanac and ephemeris).
- 2008 ● Google Maps was first released for Android and iOS after the introduction of **My Location** feature using **GPS/Assisted GPS** supplemented by wireless network and cell sites
- 2016 ● Google allowed the **access to raw measurements** through a specific Location API available on Android API-Level 24.
- 2018 ● The first **Dual Frequency GNSS smartphone**, the Xiaomi Mi 8 Pro, approached the market with the embedded Broadcom BCM47755 chipset
- 2020 ● 2° Generation Broadcom Chipsets **BCM47765 with High-Definition GPS (HDGPS)**
! Optional on-chip PVT calculations !

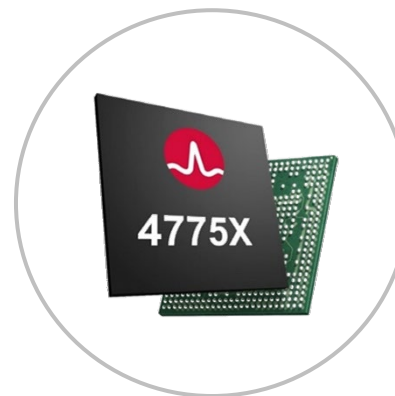
<https://www.gpsworld.com/wireless-smartphone-revolution-9183/> By Frank van Diggelen, Broadcom Corporation

GNSS Inside an Android phone



Xiaomi Mi 8 Pro

BROADCOM
BCM47755



Location APIs

- Places, Geofencing
- Fused Location Provider (FLP)
- Fit, Activity Recognition
- Nearby

Measurement/Sensor APIs

- Location
- GNSS Measurement
- GNSS Clock

RAW GNSS Measurements

FEATURES

- Integrated **multi-frequency** GNSS baseband and RF front end for reception of **GPS, GLONASS, BeiDou (BDS), Galileo (GAL)**, and SBAS satellite systems
- Support for position batching, geofencing, sensor fusion and sensor navigation

How to read data

Android 7 Location - Clock and Measurements		
ANDROID CLASS	FIELD	DESCRIPTION
GNSSClock	<i>TimeNanos</i>	GNSS receiver's internal hardware clock value in nanoseconds
GNSSClock	<i>BiasNanos</i>	Clock's sub-nanosecond bias
GNSSClock	<i>FullBiasNanos</i>	Difference between TimeNanos inside the GPS receiver and the true GPS time since 0000Z, 6 January 1980
GNSSClock	<i>DriftNanosPerSecond</i>	Clock's drift
GNSSClock	<i>HardwareClockDiscontinuityCount</i>	Count of hardware clock discontinuities
GNSSClock	<i>LeapSecond</i>	Leap second associated with the clock's time
GNSSMeasurement	<i>ConstellationType</i>	Constellation type
GNSSMeasurement	<i>Svid</i>	Satellite ID
GNSSMeasurement	<i>State</i>	Current state of the GNSS engine
GNSSMeasurement	<i>ReceivedSvTimeNanos</i>	Received GNSS satellite time at the measurement time
GNSSMeasurement	<i>AccumulatedDeltaRangeMeters</i>	Accumulated delta range since the last channel reset
GNSSMeasurement	<i>Cn0DbHz</i>	Carrier-to-noise density
GNSSMeasurement	<i>TimeOffsetNanos</i>	Time offset at which the measurement was taken in nanoseconds
GNSSMeasurement	<i>CarrierCycles</i>	Number of full carrier cycles between the satellite and the receiver
GNSSMeasurement	<i>CarrierFrequencyHz</i>	Carrier frequency at which codes and messages are modulated
GNSSMeasurement	<i>PseudorangeRateMetersperSecond</i>	Gets the Pseudorange rate at the timestamp

Fundamental to the determination of pseudorange measurements

Fundamental to the determination of pseudorange measurements

Fundamental to the determination of pseudorange measurements

Table included in the **White paper**, "Using GNSS Raw Measurements on Android Devices"

How to read data

Android 7 Location - Clock and Measurements		
ANDROID CLASS	FIELD	DESCRIPTION
GNSSClock	<i>TimeNanos</i>	GNSS receiver's internal hardware clock value in nanoseconds
GNSSClock	<i>BiasNanos</i>	Clock's sub-nanosecond bias
GNSSClock	<i>FullBiasNanos</i>	Difference between TimeNanos inside the GPS receiver and the true GPS time since 0000Z, 6 January 1980
GNSSClock	<i>DriftNanosPerSecond</i>	Clock's drift
GNSSClock	<i>HardwareClockDiscontinuityCount</i>	Count of hardware clock discontinuities
GNSSClock	<i>LeapSecond</i>	Leap second associated with the clock's time
GNSSMeasurement	<i>ConstellationType</i>	Constellation type
GNSSMeasurement	<i>Svid</i>	Satellite ID
GNSSMeasurement	<i>State</i>	Current state of the GNSS engine
GNSSMeasurement	<i>ReceivedSvTimeNanos</i>	Received GNSS satellite time at the measurement time
GNSSMeasurement	<i>AccumulatedDeltaRangeMeters</i>	Accumulated delta range since the last channel reset
GNSSMeasurement	<i>Cn0DbHz</i>	Carrier-to-noise density
GNSSMeasurement	<i>TimeOffsetNanos</i>	Time offset at which the measurement was taken in nanoseconds
GNSSMeasurement	<i>CarrierCycles</i>	Number of full carrier cycles between the satellite and the receiver
GNSSMeasurement	<i>CarrierFrequencyHz</i>	Carrier frequency at which codes and messages are modulated
GNSSMeasurement	<i>PseudorangeRateMetersperSecond</i>	Gets the Pseudorange rate at the timestamp

getTimeNanos

Local Timestamp attributed to the measurements



For battery saving or other reasons determined by the operating system, clock might have discontinuities!

Table included in the **White paper**, "Using GNSS Raw Measurements on Android Devices"

How to read data

Android 7 Location - Clock and Measurements		
ANDROID CLASS	FIELD	DESCRIPTION
GNSSClock	<i>TimeNanos</i>	GNSS receiver's internal hardware clock value in nanoseconds
GNSSClock	<i>BiasNanos</i>	Clock's sub-nanosecond bias
GNSSClock	<i>FullBiasNanos</i>	Difference between TimeNanos inside the GPS receiver and the true GPS time since 0000Z, 6 January 1980
GNSSClock	<i>DriftNanosPerSecond</i>	Clock's drift
GNSSClock	<i>HardwareClockDiscontinuityCount</i>	Count of hardware clock discontinuities
GNSSClock	<i>LeapSecond</i>	Leap second associated with the clock's time
GNSSMeasurement	<i>ConstellationType</i>	Constellation type
GNSSMeasurement	<i>Svid</i>	Satellite ID
GNSSMeasurement	<i>State</i>	Current state of the GNSS engine
GNSSMeasurement	<i>ReceivedSvTimeNanos</i>	Received GNSS satellite time at the measurement time
GNSSMeasurement	<i>AccumulatedDeltaRangeMeters</i>	Accumulated delta range since the last channel reset
GNSSMeasurement	<i>Cn0DbHz</i>	Carrier-to-noise density
GNSSMeasurement	<i>TimeOffsetNanos</i>	Time offset at which the measurement was taken in nanoseconds
GNSSMeasurement	<i>CarrierCycles</i>	Number of full carrier cycles between the satellite and the receiver
GNSSMeasurement	<i>CarrierFrequencyHz</i>	Carrier frequency at which codes and messages are modulated
GNSSMeasurement	<i>PseudorangeRateMetersperSecond</i>	Gets the Pseudorange rate at the timestamp

`getTimeNanos - (getFullBiasNanos + getBiasNanos)`

Absolute timestamp (GNSS time)

Table included in the **White paper**, "Using GNSS Raw Measurements on Android Devices"

How to determine pseudorange measurements



TRANSMISSION TIME

Transmission time in GNSS time scale

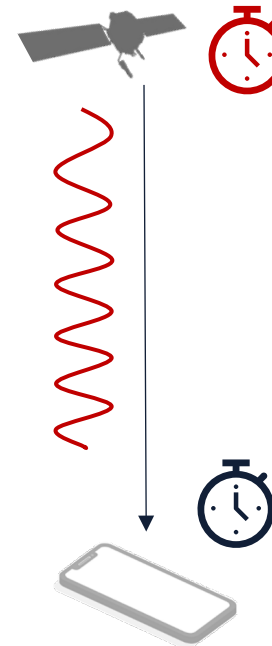
`TxTime= ReceivedSvTimeNanos`



RECEPTION TIME

Absolute Timestamp attributed to the measurements

`Rxtime= getTimeNanos - (getFullBiasNanos+getBiasNanos) -weekNumberNanos`



How to determine pseudorange measurements



TRANSMISSION TIME

Transmission time in GNSS time scale

$$TxTime = ReceivedSvTimeNanos$$



RECEPTION TIME

Absolute Timestamp attributed to the measurements

$$Rxtime = getTimeNanos - (getFullBiasNanos + getBiasNanos) - weekNumberNanos$$



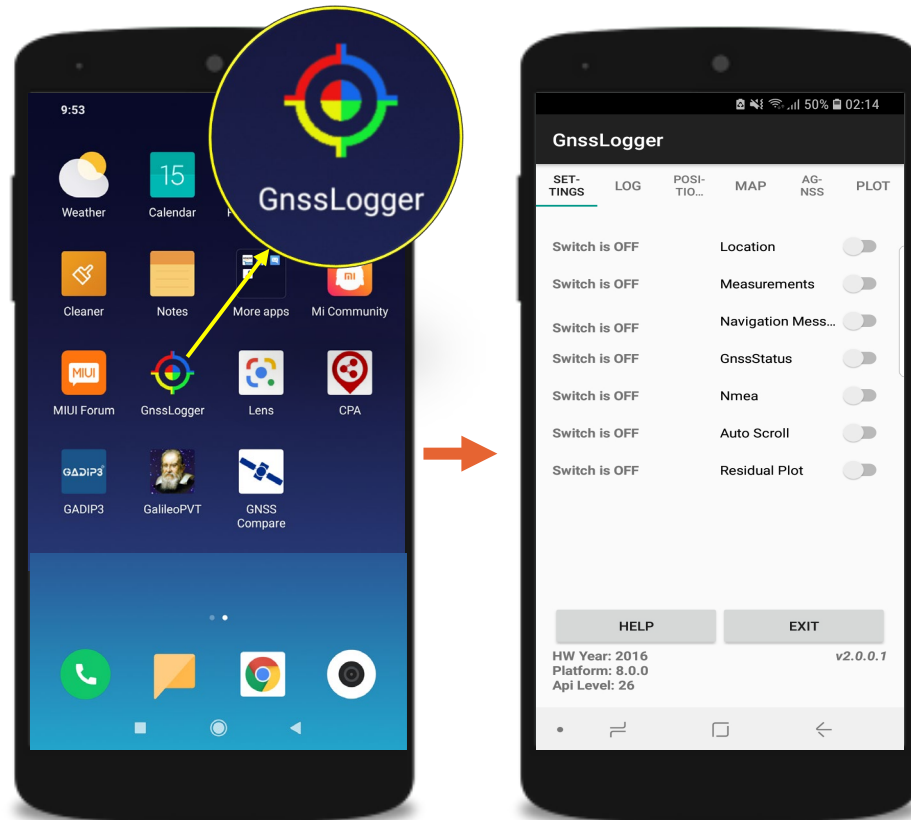
PSEUDORANGE MEASUREMENTS

Geometrical distance between satellite and receiver still affected by local clock bias

$$Raw = (Rxtime - TxTime) * c$$

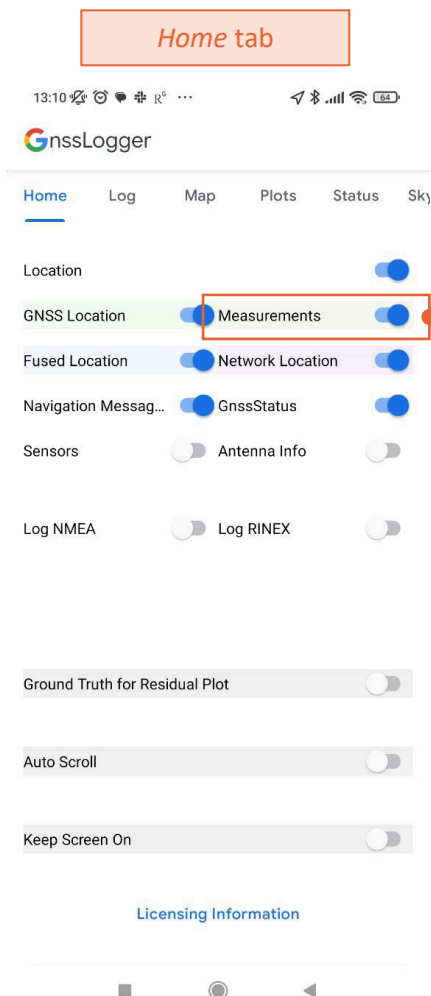
How to perform an Analysis

1) ACQUIRING & LOG MEASUREMENTS on Android Phones

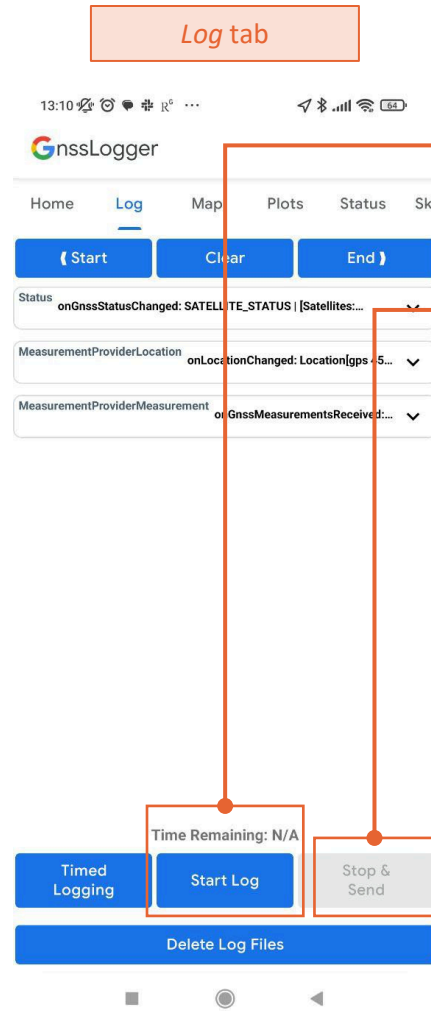


```
1 #
2 # Header Description:
3 #
4 # Version: v2.0.0.1 Platform: 8.0.0 Manufacturer: samsung Model: SM-G950F
5 #
6 # Raw,ElapsedRealtimeMillis,TimeNanos,LeapSecond,TimeUncertaintyNanos,FullBiasNanos,BiasNanos,1
7 #
8 # Fix,Provider,Latitude,Longitude,Altitude,Speed,Accuracy, (UTC) TimeInMs
9 #
10 # Nav,Svid,Type,Status,MessageId,Sub-messageId,Data (Bytes)
11 #
12 Raw,1081060985,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,7,0.0,17,937523,
13 Raw,1081060985,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,8,0.0,16431,2072
14 Raw,1081060986,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,10,0.0,16431,207
15 Raw,1081060986,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,11,0.0,17,328161
16 Raw,1081060986,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,15,0.0,39,291723
17 Raw,1081060987,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,16,0.0,16431,207
18 Raw,1081060987,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,18,0.0,16431,207
19 Raw,1081060987,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,20,0.0,16431,207
20 Raw,1081060987,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,21,0.0,16431,207
21 Raw,1081060988,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,26,0.0,16,124101
22 Raw,1081060988,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,27,0.0,16431,207
23 Raw,1081060988,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,30,0.0,16,124101
24 Raw,1081060989,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,95,0.0,32995,452
25 Raw,1081060989,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,9,0.0,16,2049899
26 Raw,1081060989,1241897000000,,,-1221297255102982384,0.0,8.751339109972509,,,0,11,0.0,32995,452
```

How to perform an Analysis: GNSS Logger App

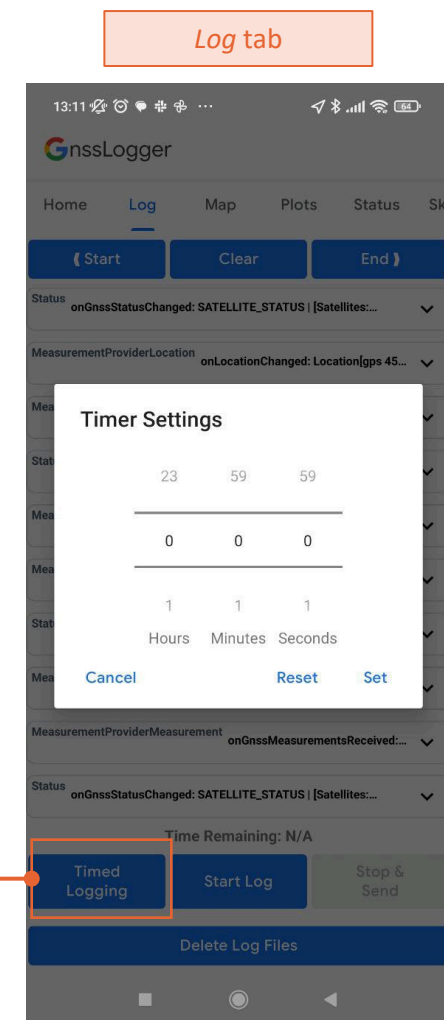


- «Measurements» **must** be switched on
- Other toggles can be on, but they are not essential



Log tab

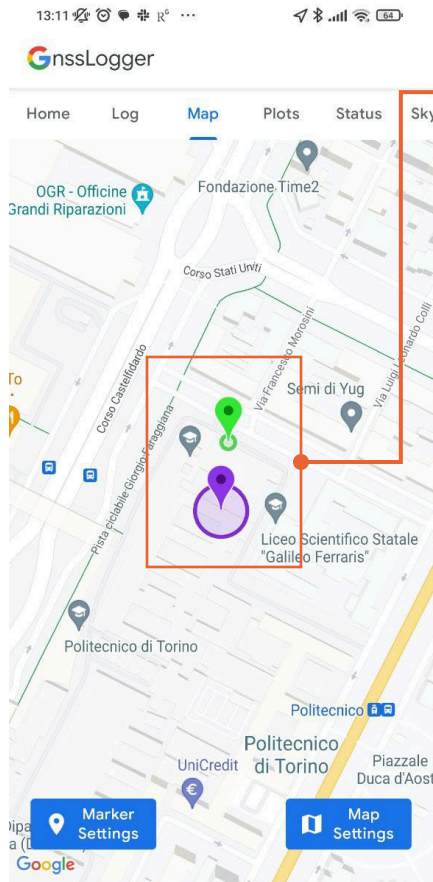
- It starts saving the log data
- Stop data logging. The app asks you were to send the log file (you should manage to get them to your PC)
- Timed logging is useful to ensure that multiple data collections have the same length (useful to compare experiments' outputs)



Log tab

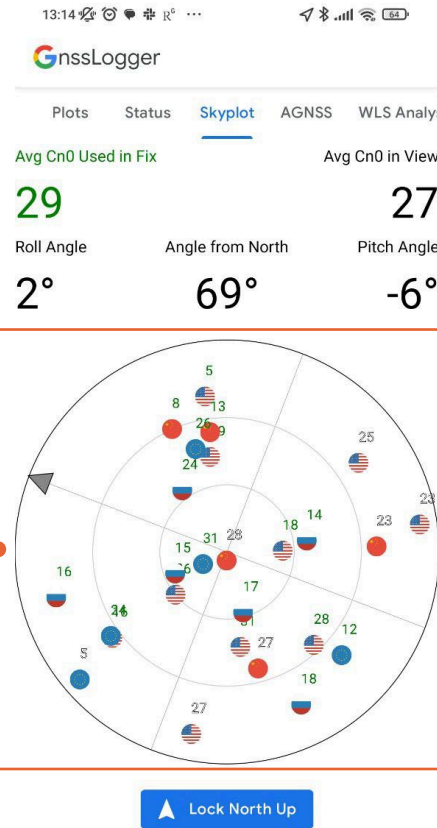
How to perform an Analysis: GNSS Logger App

Map tab



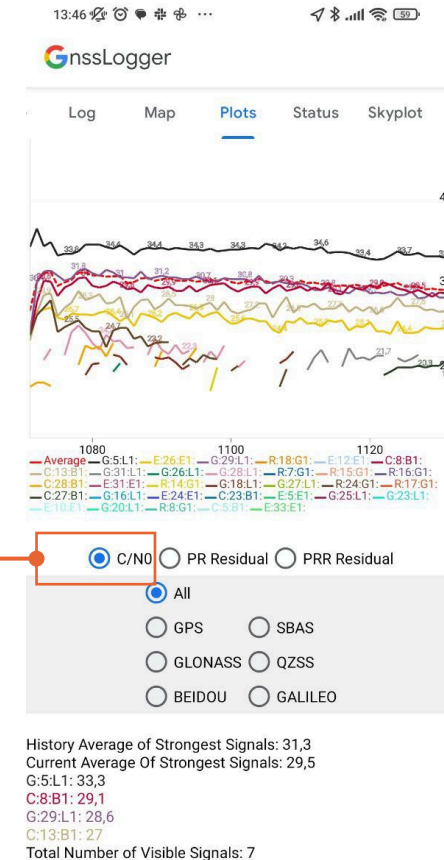
- Positions from different OS location providers are shown (GNSS, network, fused)
- The **GNSS position** (green) is different from the **network location provider position** (purple)

Skyplot tab



- A **Skyplot** shows the position of satellites in terms of azimuth and elevation
- You can also get **skyplots at a given spacetime location from** <https://www.gnssplanning.com/#/settinggs>

Plots tab



- C/N0 plots show you which satellites are visible and which are the strongest signals

Material and references



- **White paper**, “Using GNSS Raw Measurements on Android Devices”
https://www.euspa.europa.eu/system/files/reports/gnss_raw_measurement_web_0.pdf
- **GNSS Analysis MATLAB code (or app) and Android Logger APK can be downloaded from:**
GnssAnalysis[OS].zip: <https://github.com/google/gps-measurement-tools/releases>
Gnsslogger.apk: <https://github.com/google/gps-measurement-tools/releases/tag/2.0.0.1>
- **What about your phone?**
Does it provide raw GNSS measurements?: <http://g.co/gnss tools>
Does it use Galileo?: <https://www.usegalileo.eu/EN/inner.html#data=smartphone>



References:

- [1] <https://www.gpsworld.com/wirelesssmartphone-revolution-9183/>
- [2] <https://developer.android.com/reference/android/location/GnssMeasurement>
- [3] <http://gpsworld.com/google-opens-up-gnss-pseudoranges/>
- [4] S. Banville, F. Van Diggelen, “Precise GNSS for Everyone: Precise Positioning Using Raw GPS Measurements from Android Smartphones”, GPS World 27(11), November 2016. <http://gpsworld.com/innovation-precise-positioning-using-raw-gps-measurements-from-android-smartphones/>
- [5] F. Van Diggelen, “GNSS Measurements Update”, GSA Raw Measurements Workshop, Prague, 30 May 2018.
https://www.gsa.europa.eu/sites/default/files/expo/frank_van_diggelen_keynote_android_gnss_measurements_update.pdf

Lab Session 1: Raw GNSS measurements in Android devices

Laboratory on Post Processing of Raw Measurements | **LAB TASKS**

Simone Zocca, Andrea Nardin (Prof. Alex Minetto)

name.surname@polito.it



TASKS LIST



TASK

1

Download the *Google's GPS measurement tool* from the portal and test the MATLAB code by running the suggested demo datasets.

TASK

2

Observe and take detailed notes on the data observed from the generated figures with the help of the tutors. Note the behavior over time of the quantities studied in class, e.g., the relationship between satellite distance (measured via pseudorange) and C/No levels. Clarify the meaning of each observed quantity (with the help of the tutors and instructors, if required).

TASK

3

Design a routine that counts the number of visible satellites per epoch and check if conditions for estimating the receiver state are satisfied (according to formulation of multi-lateration problem we studied in class). Compute the percentage of time in which visible satellites are sufficient to achieve a PVT estimation. This is a proxy for *GNSS service availability*.

TASK

4

Investigate the use of *data filters* (e.g., C/No, Constellation) by comparing output results at **Task 2** and **Task 3** in different conditions. E.g., verify the PVT availability when C/No filter is more stringent and for different constellations.

TASKS LIST



TASK

5

Retrieve new data collections (10 minutes) using your own Android device with *GNSS Logger App* (if any) and perform raw measurements analysis. Discuss the output of the MATLAB script by testing your own data collection, and reporting the availability you evaluated at **Task 3**.

TASK

6

Verify the relationships between the relative satellite-receiver position on a skyplot (available e.g. [here](#)) and the C/No levels with their temporal trends.

Hint: Use one of the planners presented in class (links available on the portal) to check the relative position of the GNSS satellites w.r.t. to the position where the dataset was collected. Evaluate potential obstacles to direct visibility by eventually re-exploring the location.

TASK

7

Access the inner levels of the MATLAB script and determine where the essential information for calculating the PVT (purpose of next week's lab) is obtained.

Observe the structure of this information in preparation for writing an iterative Least Square solution to estimate the position and clock bias based on real data collections.

Get and run GPS-MEASUREMENT-TOOLS

- Download the modified *Google's GPS measurement tool* from the portal

- 1. Unzip **gps-measurement-tools-master** and go inside **opensource** folder
- 2. Open main script: **ProcessGnssMeasScript.m** with MATLAB
- 3. Copy your log file(s) inside **~/opensource/demoFiles** (e.g. ~/opensource/demoFiles/myLogs)
- 4. Match your folder path by updating dirName (e.g. **~/opensource/demoFiles/myLogs**)



In case of issues in the use of the enhanced Google's gps-measurement-tools-master downloaded from the portal, you can consider using the more stable version available on Google's github (it will likely need some workaround with some smartphones models)

Inside the Main Script

- Main Script: *ProcessGnssMeasScript*

Launching the main script is enough to process raw measurements and plot the results

Function to process the raw measurements and compute code Pseudoranges:

- *[gnssMeas] = ProcessGnssMeas(gnssRaw);*

Function to perform GNSS PVT:

- *gpsPvt = GpsWlsPvt(gnssMeas,allGpsEph);*
- It will be finalized in the next Lab!

Explore the output structures 'gnssMeas' and plot their fields individually.

- Try exploring the 'diff' MATLAB function on pseudoranges.

Inside the Main Script

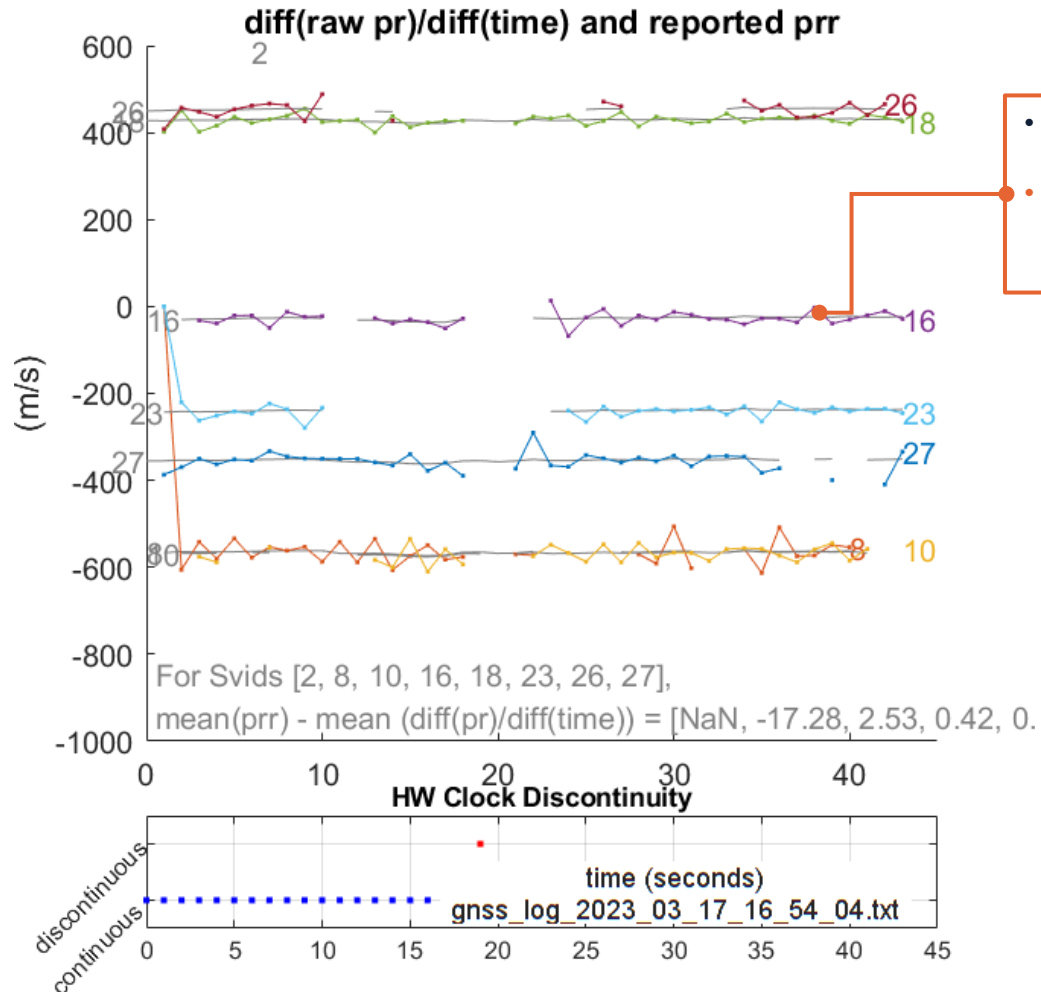
Input Parameters:

- File Name and Directory Name
- True Coordinate
 - It will be useful in next lab to compute errors
 - Write it down if known (can be inferred from google maps for instance)

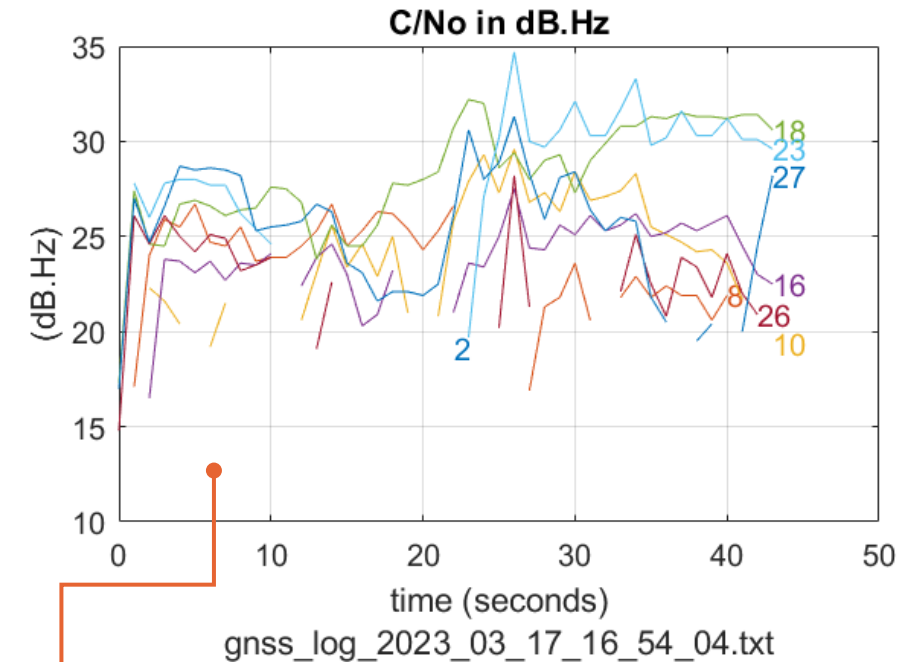
```
prFileName = 'pseudoranges_log_2016_06_30_21_26_07.txt'; %with duty cycling, no carrier phase
% prFileName = 'pseudoranges_log_2016_08_22_14_45_50.txt'; %no duty cycling, with carrier phase
% as follows
% 1) copy everything from GitHub google/gps-measurement-tools/ to
%     a local directory on your machine
% 2) change 'dirName = ...' to match the local directory you are using:
dirName = '~/Documents/MATLAB/gpstools/opensource/demoFiles';
% 3) run ProcessGnssMeasScript.m script file
param.llaTrueDegDegM = [];
```

- **Try running the script with the demo log file first!**
- Example of true coordinates in LLA (latitude, longitude, altitude) frame: [45.0621, 7.6633, 295.070]
... where is this location?

How to perform an Analysis: MATLAB analysis tool



- Derivatives can highlight sudden PR «jumps»
- How are the two lines computed? What is their difference?



- C/N0 tells you which are the strongest (and weakest) signals and therefore the potentially more (and less) accurate measurements

Inside the Main Script

- Data Filter:

dataFilter = SetDataFilter;

- Use some default filters by uncommenting them.

Example: Exclude specific satellites

```
dataFilter{end+1,1} = 'Svid';  
dataFilter{end,2}   = 'Svid ~= 4';
```

- Write your own custom filter.

Example: C/N0

```
dataFilter{end+1,1} = 'Cn0DbHz';  
dataFilter{end,2}   = 'Cn0DbHz>30';
```

- What would be the effect of these filters on the position?

Inside the Main Script



■ Acquiring the Ephemeris:

```
%% Get online ephemeris from Nasa ftp, first compute UTC Time from gnssRaw:
fctSeconds = 1e-3*double(gnssRaw.allRxMillis(end));
utcTime = Gps2Utc([],fctSeconds);
allGpsEph = GetNasaHourlyEphemeris(utcTime,dirName);
if isempty(allGpsEph), return, end
```



After running the script for the first time, unzip the downloaded ephemeris in “demoFiles” folder, if automatic unzip does not work

Inside the MATLAB Code: Glossary



<code>allRxMilliseconds:</code>	Milliseconds at which the measurements are dumped (local time);
<code>gnssMeas.FctSeconds:</code>	Fetch Time in Seconds (local time);
<code>N:</code>	Number of fetch seconds (measurements records and PVT if everything goes well);
<code>gnssMeas.ClkDCount:</code>	<code>zeros(N,1);</code>
<code>gnssMeas.HwDscDelS:</code>	<code>zeros(N,1);</code>
<code>gnssMeas.Svid:</code>	all the sv ids found in <code>gnssRaw</code>
<code>M:</code>	Number of unique Svid
<code>gnssMeas.AzDeg:</code>	Satellites Azimuth Degrees
<code>gnssMeas.ElDeg:</code>	Satellites Elevation Degree
<code>gnssMeas.tRxSeconds:</code>	Time of reception, seconds of GPS week
<code>gnssMeas.tTxSeconds:</code>	Time of transmission, seconds of GPS week
<code>gnssMeas.PrM:</code>	PseudoRange Measurements;
<code>gnssMeas.PrSigmaM:</code>	Pseudorange Standard Deviation;
<code>gnssMeas.DelPrM:</code>	DeltaPseudorangeMeasurements;
<code>gnssMeas.PrrMps:</code>	Pseudorange Rate;
<code>gnssMeas.PrrSigmaMps:</code>	Pseudorange Rate Standard Deviation;
<code>gnssMeas.AdrM:</code>	Accumulated Delta Range Measurements;
<code>gnssMeas.AdrSigmaM:</code>	Standard Deviation of the Accumulated Delta Range Measurements;
<code>gnssMeas.AdrState:</code>	Accumulated Delta Range State;
<code>gnssMeas.Cn0DbHz:</code>	C/No;

Lab 1 – Troubleshooting



Code Update for Lab 1 (GNSS Raw Measurements)

- A new code is available since March 2022 that is compatible with both Windows and Unix systems.
- In case of issues with extraction of ephemeris file, we recommend a manual extraction of the data downloaded from NASA's CCDIS service, matching the date and time of your data collection.

NOTES:

- Files logged through the **GNSSLogger App v3.0.0.1** can still break the code depending on the model of your device, chipset, or API version.
- Old logs available in **dataset_a** and **dataset_b** might require old-format ephemeris that cannot be automatically decompressed. In case you want to use them, please, extract them manually after the download.